

IDOR (Insecure Direct Object Reference) — Study Guide

IDOR has come up as a running example throughout this series — your CIA Triad guide used it to illustrate a confidentiality violation, your CWE guide mapped it to CWE-639, and your OWASP Top 10:2025 guide placed it under A01:2025 Broken Access Control. This is the deep dive: what IDOR actually is, how to find it systematically, and why it's one of the highest-value bugs in bug bounty and VAPT work despite requiring almost no technical sophistication to exploit.

1. What IDOR Actually Is

IDOR occurs when an application exposes a direct reference to an internal object — a database ID, a filename, a record key — and lets a user access or modify that object **just by changing the reference**, without properly verifying they're actually authorized to access that specific object.

```
Request: GET /account?user_id=1001 → returns YOUR account details
Attack:  GET /account?user_id=1002 → returns SOMEONE ELSE'S account details
```

The vulnerability isn't that an ID is visible — it's that the server **never checks whether the logged-in user is actually allowed to see the object behind that ID**. The application authenticates *who you are* but fails to authorize *what you're allowed to touch*.

2. Why IDOR Is Its Own Category (Not Just "Access Control")

IDOR is a specific, narrow pattern within the broader Broken Access Control category — worth distinguishing from related-but-different issues:

Vulnerability	The Flaw
IDOR	Object reference is exposed and user-controllable; server doesn't verify ownership
Privilege Escalation	User gains a higher permission <i>level</i> entirely (user → admin)
Missing Function-Level Access Control	An endpoint/feature itself has no auth check at all, regardless of object references
CSRF	A different user's <i>browser</i> is tricked into making a request — the request itself might be properly authorized

IDOR specifically hinges on **object-level** authorization failure — the user is legitimately logged in, using legitimate functionality, just pointed at someone else's data.

3. Where Object References Show Up

Direct object references appear far more often than people initially notice once you're looking for them:

```

URL parameters:      ?id=1001, ?user=jsmith, ?order_id=5502
Path parameters:     /api/users/1001, /invoices/5502
Hidden form fields:  <input type="hidden" name="account_id" value="1001">
Cookies:              session_context=user1001
Request bodies (JSON): {"user_id": 1001, "action": "view_profile"}
File references:      /uploads/user1001/resume.pdf
API responses: a GET response reveals object IDs you can then reuse in other calls

```

A frequently missed source: even when the UI never *displays* an ID directly, it often exists in the underlying API calls the frontend makes — this is exactly why proxying your browser traffic through Burp/ZAP (from your OWASP ZAP guide) matters so much for IDOR hunting; you see every request the app actually sends, not just what's rendered on screen.

4. Types of IDOR

a) Horizontal IDOR (most common)

Accessing another user's data at the **same privilege level** — user A viewing/modifying user B's data, where both are regular users.

```
GET /api/orders/5502    (yours)
GET /api/orders/5503    (someone else's order — same access
level, different owner)
```

b) Vertical IDOR

Accessing data or functionality that should require a **higher** privilege level than the attacker has.

```
GET /api/admin/users/export    ← a regular user hitting an
admin-only endpoint by
                                guessing/discovering the
URL, with no server-side
                                role check enforcing it
```

c) IDOR via Object Modification (not just viewing)

Many people mentally file IDOR as "read-only" data leakage, but it applies equally to `PUT`, `POST`, and `DELETE` operations:

```
PUT /api/profile/1001    {"email": "attacker@evil.com"}
→ if the server doesn't verify 1001 belongs to the requester,
   the attacker just changed someone else's account email
```

This category is frequently more severe than read-only IDOR, since it enables account takeover, data tampering, or destructive actions — not just information disclosure.

5. Practical Example — Full Walkthrough

1. Log in as a normal user, browse to your own profile
`GET /api/profile/1044`
Response: `{"id": 1044, "name": "You", "email": "you@test.com", "ssn": "***-**-1234"}`
2. Notice the numeric, sequential ID in the request
3. Try an adjacent ID
`GET /api/profile/1043`
Response: `{"id": 1043, "name": "Someone Else", "email": "other@test.com", "ssn": "***-**-5678"}`
4. Confirmed: IDOR. No authorization check ties the requested ID to the authenticated session — you can now enumerate every user in the system by simply iterating the ID

This last step — sequential enumeration — is exactly why **predictable, sequential IDs are themselves a risk multiplier**: even if a single IDOR instance seems minor, sequential numbering turns it into a full data dump via simple automation (`for i in {1..10000}; do curl .../profile/$i; done`).

6. Finding IDOR Systematically (Testing Methodology)

a) Map every object reference in the application first

Proxy all your traffic through Burp/ZAP while clicking through every feature of the app as a normal user — build a complete picture of every ID/reference the app uses (this is the same "manual browsing beats automated crawling" principle from your ZAP guide).

b) Create two test accounts

This is the single most important practical step — you cannot properly test IDOR with only one account, since you need a second, genuinely separate user's data to attempt to access. Register/use Account A and Account B, and deliberately try to access Account B's resources while authenticated as Account A.

c) Systematically swap every reference you find

- Numeric IDs → increment/decrement (1001 → 1000, 1002)
- UUIDs → harder to guess, but still worth testing if you can obtain another user's UUID through some other channel (e.g., it leaks in a different response, or an invite link)
- Usernames/emails as identifiers → substitute a known other username
- File paths → try adjacent/predictable filenames

d) Test read AND write operations separately

A GET might be properly protected while the corresponding PUT / DELETE on the same object isn't (or vice versa) — each HTTP method needs its own explicit test, never assume protection on one implies protection on all.

e) Test across privilege boundaries, not just between peers

Don't only swap between two regular-user accounts — also test whether a low-privilege account can reach objects/endpoints that should require admin/elevated access (vertical IDOR).

f) Check API responses for leaked references

Sometimes a legitimate API response includes another user's ID as a side effect (e.g., a "recently active users" widget, an activity feed, an autocomplete field) — these are gold, since they hand you valid IDs to test against without guessing.

7. Using Burp/ZAP to Test IDOR Efficiently

1. Proxy your browser through ZAP/Burp
2. Log in as Account A, perform an action that references an object (e.g., GET /api/orders/5502)
3. Send that request to Repeater (Burp) or the Manual Request Editor (ZAP)
4. Change the ID to a value you know belongs to Account B
5. Resend - if you get Account B's data back, you've confirmed

med IDOR

6. Use Intruder/Fuzzer to automate testing a range of IDs at scale,
confirming the pattern holds across many objects, not just one

This directly extends the Fuzzer workflow from your OWASP ZAP guide — instead of fuzzing for injection payloads, you're fuzzing the ID parameter itself and watching for successful responses to IDs that shouldn't belong to your session.

8. Distinguishing IDOR From a Non-Issue

Not every changeable ID is a vulnerability — a careful tester confirms the server actually returns **unauthorized** data, not just that the request didn't error:

False signal: changing the ID returns a 403/404 or an empty/generic response

→ the server IS checking authorization correctly, this is NOT IDOR

True signal: changing the ID returns another user's actual data, or successfully

modifies/deletes it

→ confirmed IDOR

Also worth checking: does the object legitimately belong to a *shared* context (e.g., a public product listing anyone can view by design)? Not every accessible object-by-ID is a bug — the question is always whether access **should** have been restricted and wasn't.

9. Real-World Severity Range

IDOR severity varies enormously depending on what's exposed:

Low: viewing another user's public-facing display name (already semi-public info)

Medium: viewing another user's private order history, address

High: viewing another user's PII (SSN, financial data), or modifying their account
Critical: IDOR on an admin function, or an IDOR chain leading to full account takeover (e.g., changing another user's password reset email via IDOR, then triggering a password reset)

IDOR chaining is worth knowing as a concept — a single IDOR might seem low-impact in isolation, but combined with a second flaw (like a password reset flow that trusts a client-supplied email/user ID) it can escalate dramatically. This is exactly the kind of "combine multiple minor findings into one critical chain" thinking that separates strong bug bounty reports from a list of disconnected low-severity bugs.

10. Prevention — Fixing IDOR Properly

The core fix: enforce object-level authorization on every single request that references an object, server-side, every time.

Vulnerable pattern:

```
@app.route('/api/orders/<order_id>')
def get_order(order_id):
    order = db.get_order(order_id)    # fetches by ID with NO ownership check
    return jsonify(order)
```

Fixed pattern:

```
@app.route('/api/orders/<order_id>')
def get_order(order_id):
    order = db.get_order(order_id)
    if order.user_id != current_user.id:
        abort(403)                    # explicit ownership check before returning data
    return jsonify(order)
```

Additional layered defenses:

- **Use indirect references** — instead of exposing the raw internal database ID, use a per-session or per-user mapping/lookup table (a random token that only resolves to the correct object for the correct user) — this doesn't replace proper authorization checks, but it removes the ability to simply guess/increment IDs
- **UUIDs instead of sequential integers** — makes blind enumeration impractical, though this is *obscurity*, not authorization — a UUID-based IDOR is still exploitable if the check is missing and the UUID leaks elsewhere
- **Centralize authorization logic** — a single, consistently-applied access-control function/middleware used across every endpoint, rather than each developer re-implementing ownership checks ad hoc (inconsistent implementation is exactly how some endpoints end up protected while sibling endpoints don't)
- **Default-deny** — access control should fail closed (deny by default, explicitly allow) rather than fail open — this connects directly to your OWASP Top 10:2025 guide's new A10:2025 category (Mishandling of Exceptional Conditions), since an error in the authorization check itself should never silently default to "allow"

11. Practical Lab Example

DVWA doesn't have a dedicated IDOR module by name, but the "Insecure CAPTCHA" and general low-security modules often expose similar direct-reference patterns worth testing manually.

WebGoat has a dedicated "Insecure Direct Object References" lesson set specifically designed to walk through exactly this vulnerability class step by step.

Since you've used both platforms already, WebGoat's IDOR-specific module is the

more directly applicable one to revisit **for** hands-on practice here.

Practice pattern **for** any lab:

1. Register two accounts
2. Perform an action **as** Account **A** that references an **object** by **ID**
3. Capture the request in **ZAP**
4. Swap the **ID** to something belonging to Account **B**
5. Confirm whether the server enforces ownership

12. Kill Chain / ATT&CK / CWE / OWASP — Full Mapping

Framework	Mapping
CWE	CWE-639 (Authorization Bypass Through User-Controlled Key) — a child of CWE-284 (Improper Access Control)
OWASP Top 10:2025	A01:2025 Broken Access Control
CVSS	Varies widely by data sensitivity — Privileges Required (PR) is typically Low/None, User Interaction (UI) typically None, making the exploitability metrics severe even when impact is moderate
Cyber Kill Chain	Exploitation → Actions on Objectives (often directly <i>is</i> the data exfiltration step, with no further stages needed)
MITRE ATT&CK	T1190 (Exploit Public-Facing Application)

Quick Reference Cheat Sheet

Test methodology:

1. Two accounts, always
2. Map every **object reference** (proxy everything through **ZAP**/Burp)
3. Swap IDs between accounts - test **GET**, **PUT**, **POST**, **DELETE** separately
4. Test **vertical** (low-priv → admin) **as well as horizontal**

(peer → peer)

5. Confirm actual unauthorized **DATA** returned, not just a lack of error

Where to look:

URL params, path params, hidden form fields, cookies, **JSON** bodies, file paths

Fix:

```
if object.owner_id != current_user.id: deny
    (server-side, on every request, every time - no exceptions)
```